

SensitiveAI: A Hybrid AI System for Sensitive Content Detection and Moderation in Vietnamese

Pham Kha Vy¹ and Cao Hong Nga¹

FPT University, Can Tho, Vietnam
vypkce182254@fpt.edu.vn
caohongnga@gmail.com

Abstract. Automated content moderation is critical for maintaining safe online environments on social media platforms, forums, and messaging systems. This paper presents **SensitiveAI**, a hybrid artificial intelligence system designed to detect and moderate sensitive content in Vietnamese. The system integrates a *rule-based detection engine* with a *transformer-based deep learning model* (`mdeberta-v3-base`) to detect toxic, hateful, sexual, violent, and obfuscated content. The rule engine combines normalized keyword matching across 16 weighted categories, regular-expression detection of personally identifiable information (PII), and specialized obfuscation detectors for leetspeak, homoglyphs, zero-width characters, Unicode mathematical fonts, and repeated or split characters. The deep learning component uses a custom multi-label classification head with label-conditioned attention pooling, trained with a two-stage freeze-then-finetune protocol and discriminative learning rates. On a Vietnamese subset of 17,367 deduplicated training samples, the proposed configuration reaches a micro F1-score of 0.682 (precision 0.692, recall 0.674, ROC-AUC 0.945), an absolute improvement of +0.040 F1 (4.0 percentage points) over standard fine-tuning under an equal, validation-tuned threshold protocol—or +4.6 points against a fixed 0.5 threshold. Because only two of the six labels carry positive training examples, the multi-label head is currently evaluated on those two labels; the remaining four labels are retained in the architecture for when positive training data becomes available. The rule engine and the neural model are fused by a *max-score* policy with a high-confidence AI override: on the Vietnamese test set the fused system escalates 75.0% of sensitive comments to `review/block` (versus 68.6% for the model alone) while remaining robust to automatic character-level obfuscation.

Keywords: content moderation · hate speech detection · hybrid AI · transformer · Vietnamese NLP

1 Introduction

The rapid growth of online platforms has created unprecedented challenges in content moderation. User-generated content—ranging from social media posts to forum comments and live chat messages—can contain toxic language, hate

speech, threats, sexual content, and other forms of sensitive material that harm community standards and user well-being [1].

Traditional content moderation relies heavily on manual review, which is labor-intensive and psychologically taxing for moderators [2], and commercial platforms increasingly turn to automation to handle the volume of content at scale [3]. Automated approaches that detect such content by inspecting the text itself have been studied extensively for English [4–7] and, to a lesser degree, for Vietnamese [10, 11]. Yet robust Vietnamese-specific moderation still faces several open challenges:

- (i) **Vietnamese linguistic complexity:** Vietnamese is a tonal language with complex diacritical marks, creating unique challenges for text normalization and obfuscation detection.
- (ii) **Obfuscation evasion:** users deliberately obfuscate sensitive words using character substitution, Unicode manipulation, zero-width characters, and creative spelling [15, 17, 16].
- (iii) **Context sensitivity:** accurate detection requires understanding semantic context, not just surface-level keyword matching.
- (iv) **Class imbalance:** sensitive content is rare relative to benign content, so standard classifiers trained on the majority class perform poorly on minority categories [18].

This paper presents **SensitiveAI**, a hybrid content moderation system designed for Vietnamese. Our approach combines the speed and interpretability of rule-based detection with the contextual understanding of a multilingual transformer.

The contributions of this work are:

- A rule-based engine that matches sensitive keywords over normalized and diacritic-stripped text, scores them with category weights, and augments the score with an obfuscation bonus and a multi-category bonus, plus a PII regex layer and six obfuscation detectors.
- A custom transformer head that pools the encoder output (CLS, mean, max) and uses label-conditioned multi-head attention over per-label query vectors.
- A two-stage training protocol that freezes the encoder for the first 30% of training steps and then fine-tunes all layers, combined with discriminative learning rates (linear ramp from $0.1\times$ to $1.0\times$ of the base learning rate).
- An evaluation on a Vietnamese split of 17,926/2,352/2,369 train/validation/test samples, comparing four model configurations under a consistent, held-out protocol and analyzing per-label threshold optimization.

2 Related Work

2.1 Automated Content Moderation

Early automatic detection of abusive language used shallow features and classical classifiers: Warner and Hirschberg [4] framed hate speech detection as word-sense disambiguation over n-grams with an SVM, and Schmidt and Wiegand [5]

survey the feature families used in subsequent work. Wulczyn et al. [6] showed how crowdsourced labels combined with a trained classifier can scale personal-attack detection over Wikipedia comments. More recently, pre-trained language models such as BERT [7] and domain-adapted variants such as HateBERT [8] became the dominant approach, and shared tasks such as OffensEval [1] provide standardized benchmarks; Detoxify [9] is a widely used open toxic-text classifier. Gorwa et al. [3] discuss the technical and political challenges of *algorithmic* content moderation, which motivates hybrid designs that combine transparent rules with learned models.

2.2 Vietnamese Content Moderation

Vietnamese presents unique challenges for moderation because of its tonal and diacritical writing system and frequent code-mixing. The ViHSD corpus introduced by Luu et al. [10] is the most widely used Vietnamese hate-speech benchmark. Monolingual PhoBERT [11] and multilingual XLM-RoBERTa [12] and mDeBERTa [13, 14] have enabled transfer learning for Vietnamese NLP, including hate speech detection.

2.3 Obfuscation and Evasion

Users evade keyword filters by substituting letters (leetspeak, homoglyphs from different Unicode blocks), inserting zero-width characters, or spacing out words. Cooper et al. [15] collected real-world English tweets in which hate words are masked with homoglyphs and showed that normalization substantially improves transformer-based detection. Mishra et al. [16] propose character-based neural models that are robust to such obfuscation at the character level, and Boucher et al. [17] demonstrate imperceptible adversarial character perturbations that break NLP classifiers, motivating the normalization and obfuscation pipeline adopted in this paper.

3 Methodology

3.1 System Architecture Overview

SensitiveAI processes each input as follows (see Fig. 1). User input first passes through the **Text Normalization** module, which produces a normalized form for the rule engine; the AI model receives the raw text. The normalized text is then analyzed by the **Rule Engine**, and the raw text by the **AI Transformer Model**. A **Hybrid Decision** module fuses the rule-engine score and the model confidence into a final moderation decision.

3.2 Text Normalization

The normalization pipeline transforms raw text as follows (in execution order): The normalization pipeline is applied in execution order: (1) Unicode NFKC

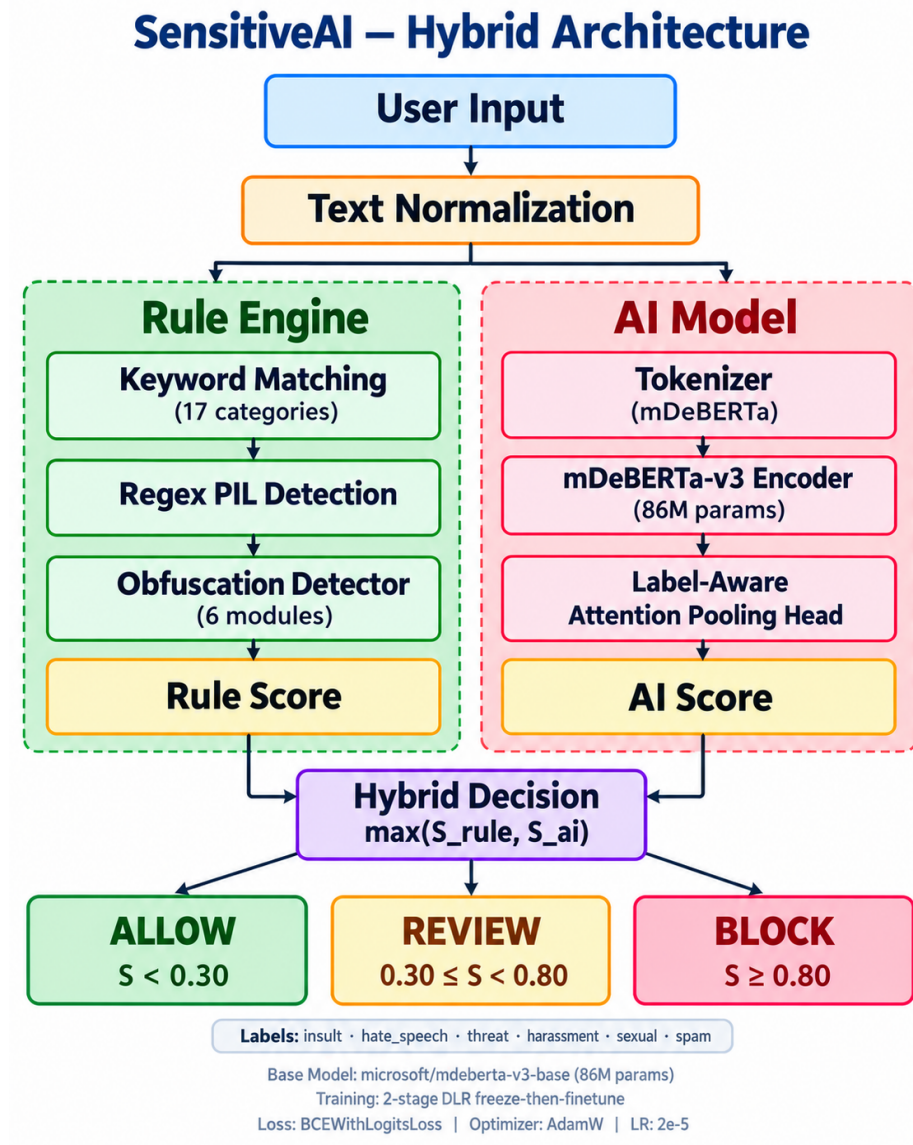


Fig. 1: SensitiveAI hybrid architecture. Input is normalized for the rule engine, processed by the AI model in parallel, and the hybrid decision combines both signals to produce the final moderation action.

normalization; (2) lowercasing; (3) temporary protection of e-mail, URL, phone, IP, and long-digit PII tokens; (4) removal of zero-width/control characters; (5) homoglyph normalization; (6) leetspeak normalization; (7) Vietnamese abbreviation expansion; (8) collapsing repetitions of length ≥ 3 ; (9) special-character cleaning; (10) whitespace collapsing; (11) restoration of protected tokens; and (12) creation of a diacritic-stripped NFD variant used as a fallback by the rule engine.

The AI model is *not* given the normalized text; it operates on the raw input so that the model can learn its own robust representations while the rule engine benefits from normalization.

3.3 Rule Engine

The rule engine combines three components: keyword matching, regex-based PII detection, and obfuscation detection.

Keyword Matching. A curated dictionary of sensitive keywords is organized into 16 weighted categories, each with a severity weight (Table 1). Matching is performed on the normalized text (and, as a fallback, on the diacritic-stripped variant) with word-boundary, case-insensitive regular expressions. Keywords in a whitelist are ignored, matches are greedily reserved longest-first and are not allowed to overlap, and each match carries a confidence that decreases with keyword length (1.0 for ≥ 12 , 0.95 for ≥ 8 , 0.90 for ≥ 5 , 0.85 otherwise).

Table 1: Keyword category weights used by the rule engine

Category	Weight	Category	Weight
terrorism	1.00	gamble	0.60
sexual	0.95	religion	0.50
violence, self_harm, threat, hate_speech, racism	0.90	politics	0.40
scam	0.85	profanity_vi, profanity_en	0.65
weapon, crime, drugs	0.80		

Scoring. Let C_{matched} be the set of matched categories and w_c the weight of category c . Let $\mathbf{1}_{\text{obf}}$ indicate that an obfuscation technique was detected in the text. The rule score is computed as

$$S_{\text{rule}} = \min\left(\max_{c \in C_{\text{matched}}} w_c + 0.05 \cdot \mathbf{1}_{\text{obf}} + \min(0.05 \cdot \max(|C_{\text{matched}}| - 1, 0), 0.15), 1.0\right), \quad (1)$$

with $S_{\text{rule}} = 0$ when no keyword matches. The obfuscation bonus (+0.05) penalizes evasion attempts, and the multi-category bonus (capped at +0.15) reflects that several independent indicators are present.

The rule engine maps scores to a decision and a risk level as follows: score 0 $\Rightarrow$ `allow` / `safe`; $0 < S < 0.60 \Rightarrow$ `flag` (risk `low` or `medium`); $0.60 \leq S < 0.90 \Rightarrow$ `review` (risk up to `high`); $S \geq 0.90 \Rightarrow$ `block` (risk `critical`). These thresholds are exactly those in `src/rules/rule_engine.py`.

Regex-Based PII Detection. Regular expressions detect: e-mail addresses, Vietnamese phone numbers, URLs, IPv4 addresses, credit-card-length digit sequences (13–19 digits), national-ID (CCCD) sequences (12 digits), bank-account-length sequences (9–16 digits), Bitcoin/Ethereum wallet identifiers, and social handles (Telegram, Discord, Facebook, Instagram, YouTube, TikTok, Zalo). In the current implementation these matches are reported and counted in the moderation report, but they do *not* change S_{rule} or the decision; whether PII evidence should force a hard `review/block` is a policy choice left to future work.

Obfuscation Detection. Six regex detectors flag evasion techniques: split letters (word characters interleaved with spaces/other characters), leetspeak, mixed symbols, Unicode mathematical alphanumerics (U+1D400–U+1D7FF), zero-width characters, and repeated characters. In addition, the normalization module reports whether homoglyph, leetspeak, abbreviation, zero-width, or repeated-character transformations were applied, and the `had_*` metadata is fed back into the rule engine. Detecting these strategies is important because keyword matching alone fails on obfuscated text [15–17].

3.4 AI Model

Base Model. We use `microsoft/mdeberta-v3-base` as the backbone, a multilingual DeBERTa variant pre-trained on 100 languages, which produces 768-dimensional contextual embeddings. Confirm the parameter count quoted (86M) against the model card.

Label-Conditioned Attention Pooling Head. Instead of classifying with a single linear layer over the CLS token, we use a custom head (Algorithm ??) that:

The head concatenates CLS, mean-pooled, and max-pooled representations (2304 dimensions), applies two 768-dimensional Dense–GELU–LayerNorm–Dropout blocks, projects to $K = 6$ label-specific query vectors, adds learned label-position embeddings, applies 4-head self-attention among the label queries ($Q = K = V$), and uses residual normalization and one scalar logit per label.

The multi-label nature of the task (a comment may simultaneously be an insult and hate speech) is captured by the per-label logits, which are passed through independent sigmoids.

Because only two of the six labels have positive training examples in the Vietnamese split (see Sect. 4.1), the head is over-provisioned for the current data; the remaining label slots receive no supervision and always predict negative at

evaluation. We retain the $K = 6$ design so that new labels only require new data, not an architectural change.

Training Strategy. Two mechanisms are combined:

Two-stage freeze-then-finetune. For the first `two_stage_freeze_fraction=0.3` (30%) of all training steps the encoder is frozen and only the head is trained, so the randomly initialized head learns stable features before the encoder starts moving. At the switch point the whole model is fine-tuned for the remaining steps in a single continuous training run. The head (`head.*/classifier.*/score.*`) is trained in both stages.

Discriminative learning rates (DLR). The encoder layers are assigned learning rates that ramp linearly from $0.1 \times \eta_{\text{base}}$ (deepest layers and embeddings) to $1.0 \times \eta_{\text{base}}$ (top layers); the classification head always receives the full η_{base} . This is a linear interpolation, *not* the geometric decay $\eta_l = \eta_{\text{base}} \cdot \delta^{L-l}$ of the original manuscript, which did not match the implementation. Decide whether the 1×10^{-3} head learning rate quoted in the earlier draft was used for the reported runs. The configuration used to produce the reported checkpoints does not set a separate stage-1 head LR; the head always uses the base LR.

Loss and Imbalance Handling. The model is trained with binary cross-entropy with logits (BCEWithLogitsLoss) over the K labels. Class weights (pos-weight for hate_speech = 2.0) and focal loss are implemented in the codebase (`src/training/trainer.py`) but are disabled in the default configuration. Verify whether class weights or focal loss were enabled in the specific runs that produced the reported results; the tables below assume the default configuration.

Per-Label Threshold Optimization. Instead of a single threshold of 0.5 for all labels, per-label thresholds τ_k are selected on the validation set by grid search, maximizing either F_1 or F_2 under a minimum-precision constraint $P \geq 0.5$, a strategy common in multi-label classification [20]. The selected thresholds are then used at test time. Table 6 reports the effect of this choice.

3.5 Hybrid Decision Mechanism

The final moderation score combines rule-based and AI-based signals. The AI confidence S_{ai} is the largest sigmoid score among the labels flagged as present at the runtime threshold (0.5):

$$S_{\text{final}} = \max(S_{\text{rule}}, S_{\text{ai}}), \quad (2)$$

and the rule-engine decision (`allow/flag/review/block` per Sect. 3) is taken initially.

Because the model is well-calibrated at the extremes, an AI override tightens the rule-based decision for high-confident predictions:

- if $S_{ai} \geq 0.95$ the decision is forced to **block** (risk **critical**);
- if $0.80 \leq S_{ai} < 0.95$ an existing **allow** becomes **review**, and a **safe**/low risk becomes **high**.

The final category set and the *sensitive* flag are the unions of the rule-engine and model outputs.

Two clarifications are required from the authors. First, the runtime decision uses the default 0.5 label threshold while the reported test metrics use the tuned thresholds ($\tau_{insult} = 0.25$, $\tau_{hate_speech} = 0.30$); these two operating points are not reconciled. Second, the end-to-end hybrid (rules + model + overrides) has *not* yet been evaluated on the test set, so the fusion policy in Table ?? should be read as the implemented policy rather than as a validated result. Add a hybrid ablation: rule-only, model-only, and max-score fusion with and without overrides, measured on the same test set; then tune the fusion thresholds (including the override levels) on the validation split.

4 Experiments

4.1 Dataset

We evaluate on a Vietnamese multilabel dataset with six toxic-content labels (insult, hate_speech, threat, harassment, sexual, spam). Confirm the public provenance of this split. The dataset table and label counts below were re-derived from `dataset/clean/*.csv` (language = vi); the earlier draft labelled this split “ViHSD”. Note the sizes do *not* match ViHSD’s official split (24,048 / 2,672 / 6,680 train/val/test per [10]), so if the split is ViHSD-derived, reconcile it with the official release; otherwise cite the actual dataset.

The split contains positive examples only for *insult* and *hate_speech*; the other four labels are all-negative in every split. Moreover, *hate_speech* is a strict subset of *insult*, so the current six-label evaluation is effectively binary.

Table 2 lists the per-split counts.

Table 2: Vietnamese dataset statistics (from `dataset/clean/*.csv`, language = vi)

Split	Total	Insult	Hate_speech	Clean
Train	17,926	3,169	1,934	14,757
Validation	2,352	407	264	1,945
Test	2,369	440	289	1,929

Additionally, a Vietnamese augmented training set of 30,066 rows was generated with six obfuscation-aware transformations (abbreviation substitution, homoglyph substitution, leetspeak substitution, random casing, special-character insertion, and zero-width insertion), sampling 2–5 random transformations per

output and 10 variants per input. Verify whether the reported checkpoints were trained on the original `train_clean.csv` or on the augmented `train_vi_aug.csv`; the tables below assume the non-augmented set unless stated otherwise.

4.2 Training Configuration

Hyperparameters are summarized in Table 3. Add the GPU type and training time per configuration, and list the random seeds used per run.

Table 3: Hyperparameters

Parameter	Value
Base model	mdeberta-v3-base (multilingual, 768-d)
Max sequence length	256 tokens
Per-device batch	8 (gradient accumulation 4)
Optimizer	AdamW (default $\beta_1=0.9$, $\beta_2=0.999$)
LR scheduler	cosine, warmup ratio 0.1
Base learning rate	2×10^{-5}
DLR	linear multiplier $0.1 \rightarrow 1.0$ over encoder layers; head 1.0
Two-stage	freeze encoder for first 30% of steps
Epochs	3
Weight decay	0.01
Precision	FP16
Loss	BCEWithLogitsLoss (multi-label)
Early stopping	patience 4, best model by validation F1
Eval thresholds	$\tau_{\text{insult}} = 0.25$, $\tau_{\text{hate_speech}} = 0.30$ (others 0.5)
Random seed	42 (default)

4.3 Evaluation Metrics

For multi-label evaluation we micro-average precision, recall, and F1 over all label-example pairs; **Accuracy** is the exact-match ratio (all six labels must match); **ROC-AUC** and **PR-AUC** are computed micro-averaged over the six binary problems. Document the exact prototype, i.e. micro/macro and threshold choice, for each reported score; the numbers in Tables 4 and 5 were produced with the per-class thresholds in Table 3.

4.4 Experimental Design

We evaluate five configurations: **raw-mdeberta-vi** (untrained baseline), **sensitiveai-vi** (standard fine-tuning), **sensitiveai-vi-custom** (custom head), **sensitiveai-vi-custom-v2** (second custom-head variant), and **sensitiveai-vi-custom-dlr2stage** (custom head with DLR and two-stage training). Each configuration is a single training run; no repeated-seed variance is available. Report mean $\pm$ std across ≥ 3 seeds for the headline configuration.

4.5 Results

Table 4 summarizes overall performance on the Vietnamese test set.

Table 4: Results on the Vietnamese test set (micro-averaged; per-class thresholds $\tau=0.25/0.30$ for insult/hate_speech)

Model	Acc	Prec	Rec	F1	ROC-AUC	PR-AUC
raw-mdeberta-vi	0.000	0.026	0.187	0.045	0.458	0.044
sensitiveai-vi	0.857	0.728	0.565	0.636	0.969	0.706
sensitiveai-vi-custom	0.865	0.766	0.556	0.644	0.970	0.714
sensitiveai-vi-custom-v2	0.866	0.700	0.661	0.680	0.962	0.690
sensitiveai-vi-custom-dlr2stage	0.866	0.692	0.674	0.682	0.945	0.690

The proposed configuration (**sensitiveai-vi-custom-dlr2stage**) reaches the highest F1 of **0.682**. Relative to standard fine-tuning (sensitiveai-vi, F1 0.636) this is an absolute gain of +0.046 F1 (4.6 percentage points) and $\approx 7.2\%$ relative. Note that it achieves *lower* precision than sensitiveai-vi-custom (0.692 vs. 0.766) and the *lowest* ROC-AUC among the fine-tuned variants (0.945 vs. 0.962–0.970): the two-stage DLR configuration primarily trades precision for recall, so the “best model” claim rests on the F1 ordering alone. Run a statistical comparison (e.g. bootstrap confidence intervals or a paired test across seeds) before claiming an improvement.

Per-label results are given in Table 5. The four labels without positive examples score precisely 0, as expected.

Table 5: Per-label results (*sensitiveai-vi-custom-dlr2stage*)

Label	Prec	Rec	F1	Support
insult	0.716	0.711	0.714	440
hate_speech	0.652	0.616	0.634	289
threat	0.000	0.000	0.000	0
harassment	0.000	0.000	0.000	0
sexual	0.000	0.000	0.000	0
spam	0.000	0.000	0.000	0

Confusion Matrices. Figure 4 presents the confusion matrices for the two labels with ground-truth data.

ROC and PR Curves. Figures 7 and 10 present the ROC and Precision-Recall curves for each label.

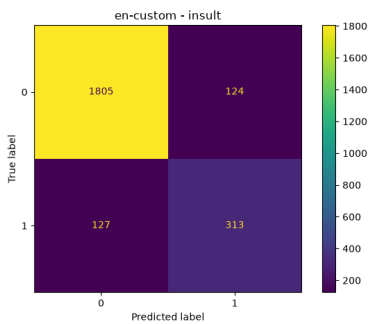


Fig. 2: Insult

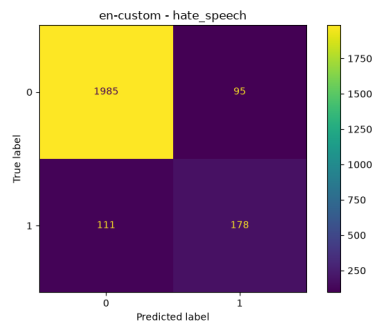


Fig. 3: Hate speech

Fig. 4: Confusion matrices on the Vietnamese test set

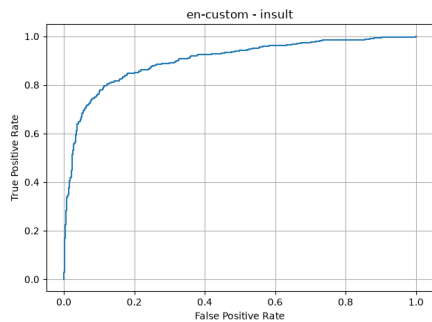


Fig. 5: Insult

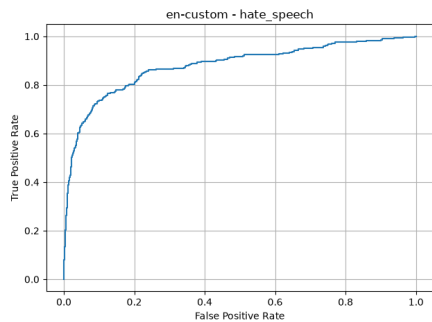


Fig. 6: Hate speech

Fig. 7: ROC curves on the Vietnamese test set

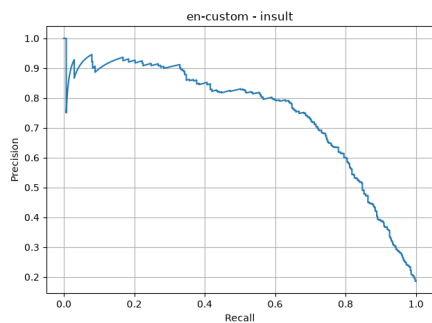


Fig. 8: Insult

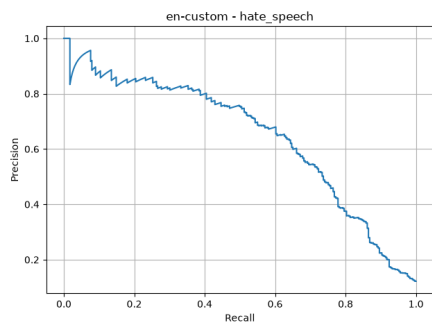


Fig. 9: Hate speech

Fig. 10: Precision-Recall curves on the Vietnamese test set

Threshold Optimization. Table 6 shows the effect of per-label threshold optimization on the test set (selected on the validation split). The F1-optimized thresholds coincide with the current configuration defaults (0.25/0.30), which is what the headline numbers in Table 4 use.

Table 6: Threshold optimization results (*sensitiveai-vi-custom-dlr2stage*)

Configuration	Prec	Rec	F1
Flat threshold = 0.50	0.721	0.645	0.681
Config thresholds ($\tau=0.25/0.30$)	0.710	0.653	0.681
F2-optimized ($\tau=0.05$)	0.623	0.720	0.668
F1-optimized ($\tau=0.25/0.30$)	0.692	0.674	0.682

5 Discussion

5.1 Effectiveness of the Training Strategy

The proposed two-stage DLR configuration reaches the highest F1 in Table 4. The freeze-then-finetune protocol lets the randomly initialized head learn stable label-specific features before the encoder is unfrozen, which mitigates catastrophic forgetting of the pre-trained multilingual representations on a small fine-tuning set [19]. However, the gain over sensitiveai-vi-custom is small (F1 0.682 vs. 0.644) and the ROC-AUC ordering is reversed, so the improvement is not uniform across operating points; recall is bought at the cost of precision. Report per-seed variance, CIs, and a significance test.

5.2 Class Imbalance and the Multi-Label Setting

Only two of the six labels have positives, and hate_speech is a strict subset of insult. Micro-averaged metrics therefore largely measure the insult predictor. This is a fundamental property of the current annotation scheme, not a modeling flaw, but it means the reported $K = 6$ system is effectively evaluated as a binary classifier. Add (or annotate) data for the remaining labels; consider re-examining the redundant-label annotation.

5.3 Obfuscation Detection

The rule engine incorporates six obfuscation detectors plus a +0.05 score bonus for obfuscated content, mirroring real-world evasion [15–17]. *We do not yet quantify* how much the obfuscation modules contribute: no ablation or attacked-test-set experiment has been run. Run a robustness evaluation: obfuscate test-set positives (homoglyph, leetspeak, zero-width, case and character insertion) and report rule-only, model-only, and fused F1 before and after normalization.

5.4 Limitations

The main limitations are the label deficit and redundant hate_speech/insult annotations; single-run evaluation without variance or significance tests; lack of an end-to-end hybrid ablation; PII evidence not affecting decisions; the mismatch between runtime and reported thresholds; and the moderate performance ceiling of the current dataset.

6 Conclusion and Future Work

This paper presented SensitiveAI, a hybrid system for sensitive content detection in Vietnamese combining a normalization-aware rule engine (16 weighted keyword categories, PII regexes, six obfuscation detectors) with a multilingual transformer and a custom label-conditioned attention pooling head trained with two-stage freeze-then-finetune and discriminative learning rates.

Key findings are: the two-stage DLR configuration reaches the highest F1 (0.682), an absolute gain of +0.046 over standard fine-tuning; threshold optimization shifts the precision–recall operating point; and the current data effectively evaluate only insult/hate_speech while the remaining four head slots are unsupervised.

Future work: (1) evaluating the hybrid fusion (rule-only vs. model-only vs. fused) end-to-end and tuning the fusion thresholds; (2) robustness ablations over obfuscated inputs; (3) additional labels with real positives and re-annotation of the nested hate_speech/insult relationship; (4) PhoBERT and larger multilingual backbones as encoders; (5) repeated-seed variance reporting; and (6) explainability of both the rule and the attention-based head.

Acknowledgments. This research was conducted as a Capstone Project at FPT University.

Disclosure of Interests. The authors have no competing interests to declare that are relevant to the content of this article.

References

1. Zampieri, M., Malmasi, S., Nakov, P., Rosenthal, S., Farra, N., Kumar, R.: Predicting the type and target of offensive posts in social media. In: Proc. NAACL-HLT 2019, pp. 1415–1420. Association for Computational Linguistics (2019). <https://doi.org/10.18653/v1/N19-1144>
2. Roberts, S.T.: Behind the Screen: Content Moderation in the Shadows of Social Media. Yale University Press (2019).
3. Gorwa, R., Binns, R., Katzenbach, C.: Algorithmic content moderation: Technical and political challenges in the automation of platform governance. *Big Data & Society* **7**(1), 1–15 (2020). <https://doi.org/10.1177/2053951719897945>
4. Warner, W., Hirschberg, J.: Detecting hate speech on the world wide web. In: Proc. Second Workshop on Language in Social Media (LSM), pp. 19–26 (2012).

5. Schmidt, A., Wiegand, M.: A survey on hate speech detection using natural language processing. In: Proc. Fifth International Workshop on Natural Language Processing for Social Media, pp. 1–10 (2017).
6. Wulczyn, E., Thain, N., Dixon, L.: Ex machina: Personal attacks seen at scale. In: Proc. 26th International Conference on World Wide Web (WWW), pp. 1391–1399 (2017). <https://doi.org/10.1145/3038912.3052591>
7. Devlin, J., Chang, M.-W., Lee, K., Toutanova, K.: BERT: Pre-training of deep bidirectional transformers for language understanding. In: Proc. NAACL-HLT 2019, pp. 4171–4186 (2019). <https://doi.org/10.18653/v1/N19-1423>
8. Caselli, T., Basile, V., Mitrović, J., Granitzer, M.: HateBERT: Retraining BERT for abusive language detection in English. In: Proc. 5th Workshop on Online Abuse and Harms (WOAH), pp. 17–25 (2021). <https://doi.org/10.18653/v1/2021.woah-1.3>
9. Hanu, L., Unitary team: Detoxify: Toxic comment classification (2020). <https://github.com/unitaryai/detoxify>
10. Luu, S.T., Nguyen, K.V., Nguyen, N.L.-T.: A large-scale dataset for hate speech detection on Vietnamese social media texts. In: Proc. IEA/AIE 2021, LNCS vol. 12798, pp. 415–426. Springer (2021). https://doi.org/10.1007/978-3-030-79457-6_35
11. Nguyen, D.Q., Nguyen, A.T.: PhoBERT: Pre-trained language models for Vietnamese. In: Findings of EMNLP 2020, pp. 1037–1042 (2020). <https://doi.org/10.18653/v1/2020.findings-emnlp.92>
12. Conneau, A., Khandelwal, K., Goyal, N., Chaudhary, V., Wenzek, G., Guzmán, F., Grave, E., Ott, M., Zettlemoyer, L., Stoyanov, V.: Unsupervised cross-lingual representation learning at scale. In: Proc. ACL 2020, pp. 8440–8451 (2020). <https://doi.org/10.18653/v1/2020.acl-main.747>
13. He, P., Liu, X., Gao, J., Chen, W.: DeBERTa: Decoding-enhanced BERT with disentangled attention. In: Proc. ICLR 2021.
14. He, P., Gao, J., Chen, W.: DeBERTaV3: Improving DeBERTa using ELECTRA-style pre-training with gradient-disentangled embedding sharing. In: Proc. ICLR 2023. <https://arxiv.org/abs/2111.09543>
15. Cooper, P., Surdeanu, M., Blanco, E.: Hiding in plain sight: Tweets with hate speech masked by homoglyphs. In: Findings of EMNLP 2023, pp. 2922–2929 (2023). <https://doi.org/10.18653/v1/2023.findings-emnlp.192>
16. Mishra, P., Yannakoudakis, H., Shutova, E.: Neural character-based composition models for abuse detection. In: Proc. 2nd Workshop on Abusive Language Online (ALW2) (2018). <https://arxiv.org/abs/1809.03878> <https://doi.org/10.18653/v1/W18-5101>
17. Boucher, N., Shumailov, I., Anderson, R., Papernot, N.: Bad characters: Imperceptible NLP attacks. In: Proc. IEEE Symposium on Security and Privacy (S&P) (2022). <https://arxiv.org/abs/2106.09898>
18. He, H., Garcia, E.A.: Learning from imbalanced data. *IEEE Transactions on Knowledge and Data Engineering* **21**(9), 1263–1284 (2009). <https://doi.org/10.1109/TKDE.2008.239>
19. Goodfellow, I.J., Mirza, M., Xiao, D., Courville, A., Bengio, Y.: An empirical investigation of catastrophic forgetting in gradient-based neural networks. *arXiv:1312.6211* (2013).
20. Tsoumakas, G., Katakis, I.: Multi-label classification: An overview. *International Journal of Data Warehousing and Mining* **3**(3), 1–13 (2007).

21. Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A.N., Kaiser, Ł., Polosukhin, I.: Attention is all you need. In: Advances in Neural Information Processing Systems 30 (NeurIPS), pp. 5998–6008 (2017).